

Stage Changes with Git Add

Git Repository Basics

Learning objective

Select changes for the next commit by adding them to Git's staging area.

Core idea

The staging area is a preparation space for the next commit. **git add** does not permanently save history; it chooses the current version of a file to include in the next commit.

Commands to know

Command / pattern	Purpose
<code>git add notes.txt</code>	Stage one file
<code>git add src/</code>	Stage changes inside a folder
<code>git add .</code>	Stage all current changes in this folder
<code>git restore --staged notes.txt</code>	Remove a file from staging

Step-by-step workflow

- 1. Edit or create a file.
- 2. Check the change using **git status**.
- 3. Stage only the intended file with **git add filename**.
- 4. Check **git status** again before committing.

Common mistakes

- Using **git add .** without reviewing all changed files.
- Thinking staged changes are already committed.
- Editing a file after staging it but forgetting to stage the newer version.

Practice task

Modify two files, stage the first one only, and verify the result with **git status**. Then unstage it safely.

Quick check

What is the difference between staging a change and committing it?

Tip: Run **git status** before and after important Git commands so you can observe what changed.